State, Not Syntax: Harness Optimization for Small-Model Multi-Turn Tool Calling

Sanskar Thapa

Master's thesis draft (presentation end of November 2026)

`https://github.com/sskarz/Thesis`

October 2026

Abstract

Assistants that *act* emit *tool calls*—a function name plus arguments—that a surrounding program, the *harness*, executes. Running this loop locally with a small language model keeps data private and costs nothing per request, but small models fail when a task needs several dependent calls. We study why with Gemma-4-E2B-it on a MacBook Pro, using the 200 multi-turn base tasks of the Berkeley Function Calling Leaderboard (BFCL v4), which grade each task by the final state of a simulated backend. The failure is *state*, not *syntax*: the model scores 83.1% on single-call selection but completes only 27.0% of multi-turn tasks through the native tool API, and two decoders that force syntactically valid calls score lower still (10.5% and 13.5%). Changing only the harness helps. With calls written as JSON text the model reaches 30.5%; adding a harness-written state summary, a stop rule for repeated calls and a four-line status card reaches 77/200 (38.5%). This gain of 8.0 percentage points is suggestive rather than established (paired $p = 0.023$ uncorrected; not significant after correcting for multiple comparisons), and it matches a five-sample majority vote (36.5%) at about a seventh of the generation time. Fine-tuning did not reach that level (best supervised fine-tune 15.0%, preference tuning 26.5%), and a control fine-tune that should change nothing points to the training data, rather than the training pipeline, as the main suspect.

1 Introduction

To check a calendar or lock a car, a language model emits a *tool call*: a function name plus arguments. A surrounding program, the *harness*, runs the call against a *backend* (here a simulated vehicle or filesystem) and feeds the result back to the model (Figure 1). Real tasks chain several dependent calls, which we call *multi-turn* tool calling: look up a value, use it, check the outcome, and know when to stop.

Running this loop locally with a small model keeps private data on the device, works offline and has no per-request cost. The catch is accuracy. We study Gemma-4-E2B-it [1] ("E2B": about two billion effective parameters; "it": instruction-tuned) on BFCL v4 [2], served with the llama.cpp inference engine [3] on a MacBook Pro. Through the native tool interface it completes 27.0% of 200 multi-turn tasks.

Is the model failing to *write* calls or to *choose* them? Section 4 shows it is mostly the latter: tasks fail because a needed tool never ran or the backend ended in the wrong state. We therefore treat the problem as *harness optimization*. The weights stay fixed; we change the loop around the model: how calls are parsed, what state the harness records and shows back, when it ends a turn, and how the next step is framed. Two of these levers, a state summary and a status card, reach the

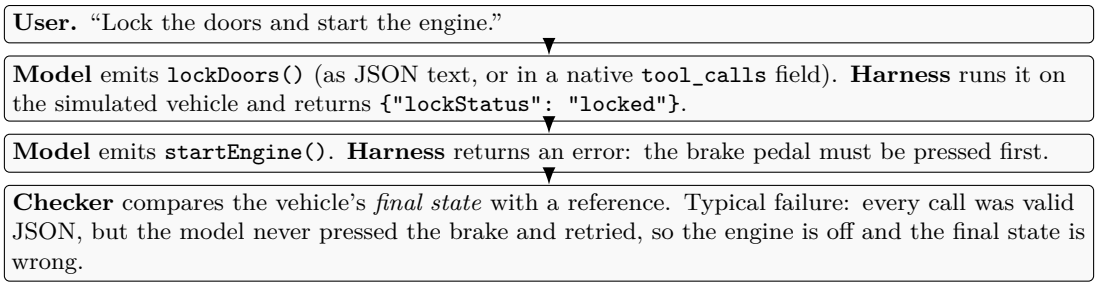


Figure 1: The multi-turn tool-calling loop studied in this paper. Grading uses the final state of the simulated backend; whether the calls parse is not enough. Illustrative, not a scored task.

model as text in its context, so the harness acts partly through the context; both are computed from executed results. We compare harness control with two other families of fixes: forcing syntactically valid output, and fine-tuning the weights. Our contributions:

1. **A failure anatomy within one model: state, not syntax.** With the same model, quantization and server, single-call accuracy is 83.1% and multi-turn accuracy 27.0%. Its failures are mostly under-calls (required tools that never ran) and wrong final states, and decoders that force valid calls lower accuracy to 10.5% and 13.5% (Section 4).

2. **A training-free harness ladder with paired statistics.** With greedy (deterministic) decoding for every harness configuration, JSON-text calls reach 30.5%, a three-part minimal recipe 35.5% and the full stack 38.5% (77/200). The full stack's gain of 8.0 percentage points (pp) over JSON text is suggestive ($p = 0.023$ uncorrected; not significant after correcting for multiple comparisons), and it matches a five-sample majority vote (36.5%) at about 1/7 of the generation time (Section 5).

3. **A controlled negative result on fine-tuning a 2B tool caller.** The best supervised fine-tune scores 15.0%, significantly below JSON text even after correction; preference tuning scores 26.5%. Models trained on a larger rebuilt data mix fail a 20-task screening check (1–2 of 20, against 17 for the untuned model), while a control fine-tune stays close to the untuned model (15 of 20), which points to the data mix rather than the training pipeline (Section 6).

2 Background and Related Work

Benchmark. The Berkeley Function Calling Leaderboard (BFCL) evaluates language models as API callers [2, 4]. Its single-turn categories check one emitted call against a reference with an abstract-syntax-tree (AST) matcher. Its multi-turn categories run the model in an agent loop against simulated backends (for example VehicleControl and GorillaFileSystem, a sandboxed Unix-like filesystem) and compare the backends' final state with a reference. A *turn* is one user request and everything the model does before handing control back; each multi-turn task has several turns. We use the *multi-turn base* split (200 tasks, all tools available). Accuracy is the fraction of the 200 tasks whose final state matches; it is not the public overall BFCL score.

Output channel. With *native function calling* (NFC), the server receives a tool list and the model returns structured `tool_calls`, as in most production APIs. With *instruction-followed function calling* (IFFC), the prompt asks the model to write its calls as a JSON list in ordinary

text, which the harness parses. Taheri et al. report that small models follow such JSON instructions more reliably than native tool APIs on single-turn BFCL [5]; we use their single-model variant.

Constrained decoding. Grammar-constrained generation restricts each next token so that output is always valid, for example to a JSON schema or a list of tool names [6, 7]; Yu et al. constrain tool names with a token-level automaton [8]. Format restrictions can also reduce answer quality [9]. Section 4 tests whether forcing validity helps in a multi-turn loop.

Harness-side control and test-time compute. Context-compression methods summarize long histories [10]; our state summary is instead written by the harness from executed results. Typed observe–orient–decide–act (OODA) controllers structure agent steps [11]; our status card borrows only the name. Guided templates add a reasoning scaffold before calls [12]. Sampling several answers and voting is a standard way to buy accuracy with compute [13]; our vote of five is that baseline.

Training for tool use. Specialist small callers such as xLAM [14] and Hammer [15] are trained on synthetic tool data such as ToolACE [16] and APIGen [17]. *Supervised fine-tuning* (SFT) trains the model to reproduce target responses; we use low-rank adapters (LoRA) [18], small trainable matrices added to frozen weights. *Direct Preference Optimization* (DPO) trains on pairs of preferred and rejected responses [19, 20]. Terms used in this paper are collected in a glossary (Appendix C).

3 Setup and Methods

Model and serving. All results use Gemma-4-E2B-it, quantized with Q4_K_M (a 4-bit weight format of llama.cpp), served by `llama-server` (llama.cpp build 8111) with Metal acceleration on a MacBook Pro (Apple M4 Pro, 48 GB). Decoding is greedy (temperature 0, so outputs are deterministic) except for majority voting. Most runs used four parallel server slots (requests the server processes concurrently) and a 65,536-token context; the native-API baseline used eight slots, a serving difference that could affect its results.

Harness interventions. All leave the weights untouched and use the same checker (Appendix A gives exact rules and examples). By design, the state summary targets lost state, the stop rule targets runaway repetition, and the status card targets the choice between acting and abstaining.

- **JSON text (IFFC)** instead of the native tool API (NFC); the harness parses the calls.
- **State summary.** After each executed call, the harness writes a short `[STATE]` snapshot of facts from real results (current directory, known files, last calls) into the conversation.
- **Stop on repeats.** The harness ends the current turn when the model repeats its previous set of calls, or when it re-proposes calls already executed in this turn after at least six tool steps. An *ungated* variant drops the six-step minimum.
- **Status card.** A four-line card (goal, known facts, last call and outcome, and an advisory next move: call, ask, final or refuse) is prefixed to the latest user message. The harness fills it deterministically from the conversation and executed results; it never sees reference answers. JSON text + stop on repeats + card is our *minimal recipe*.
- **Full stack (headline).** JSON text + state summary + stop on repeats + status card, greedy.

- **Vote of five.** On top of the state summary, at each step the harness samples five candidate calls at temperature 0.7 and executes the most common one. This is the compute-heavy comparison point.
- **Forced-valid decoding.** *Constrained decoding* restricts output in two passes (tool names, then arguments by schema); a *tool-name automaton* constrains only the name, inspired by [8].

Training. We train LoRA adapters on Gemma-4-E2B-it with MLX [21], Apple's machine-learning framework, merge them into the weights, quantize to Q4_K_M and evaluate with JSON text. Early mixes combine ToolACE dialogues with sampled xLAM data; a later *rebuilt mix* (30,000 rows) combines about 45% newly simulated, execution-checked episodes with cleaned public data. We also train one DPO model. Because full 200-task runs are slow, trained models first face a fixed 20-task *gate* drawn from the same split and run with the status card; the bar is 17/20, the untuned model's score with the card. A smaller 10-task regression check is described in Appendix B.1.

Error labels. Each failed task gets one primary label from the checker's message: *empty turn* (no call on a turn that needed one), *under-call* (a required tool never ran), or *state mismatch* (tools ran but the backend's final state differs from the reference). Output the harness cannot parse counts as no call, so a parse failure can only appear as an empty turn.

Statistics. We report Wilson 95% intervals [22], which stay accurate for small counts. For paired comparisons on the same 200 tasks we use exact two-sided McNemar tests [23], which use only the tasks on which two conditions disagree. Eight comparisons form the family for multiple-comparison correction; four of them involve the full stack, so the family was fixed after that configuration was built. We report uncorrected p-values and note where correcting for the eight comparisons (Holm's method [24], which keeps the chance of any false positive among them at 5%) changes the conclusion.

4 Where Small Models Fail

One call is easy; a chain is not. On single-call BFCL categories (simple, multiple, irrelevance, Java and JavaScript; 990 cases), Gemma-4-E2B-it through the native API scores 823/990 (83.1%, interval 80.7–85.3%; Table 4). On multi-turn tasks, the *same model, quantization and server* completes only 54 of 200 (27.0%). The two settings differ in grading as well as length, but the gap is consistent with carrying a chain of calls through a changing state, rather than choosing one call, being the bottleneck.

Most failures are about the world. Figure 2 breaks down the failed tasks (full counts in Table 2). For every JSON-text condition, state mismatch is the largest bucket (53–109 tasks). With plain JSON text it is followed by empty turns (41) and under-calls (38); the native API has more empty turns (59) and far fewer under-calls (19). Harness control mainly removes empty turns: the full stack has 20, against 41 for JSON text, while its state mismatches stay at 68. Because parse failures can only appear as empty turns, most of what remains is not a formatting problem.

Forcing valid syntax hurts. If failures were mostly malformed output, guaranteeing valid output should help. It does the opposite. Two-pass constrained decoding scores 21/200 (10.5%; interval 7.0–15.5%), 20.0 pp below JSON text, with 70 under-calls and 95 state mismatches. The tool-name

automaton scores 27/200 (13.5%; 9.4–18.9%), with state mismatch rising to 102. Our two-pass decoder approximates, rather than reproduces, full grammar engines [7], but the effect is large and in the wrong direction: a valid call is not a correct next step.

5 What Helps: Harness-Side Control

If the bottleneck is choosing the next step, the harness, which sees every executed result, is a natural place to intervene. Figure 3 shows the results. Every change is greedy except the vote of five, and none touches the weights; exact counts and intervals for every condition are in Table 4 (Appendix C).

JSON text over the native API (+3.5 pp). Writing calls as JSON text scores 61/200 against 54/200 natively ($p = 0.32$, not significant). Part of the gap may come from serving, and a guided template on the native API ties JSON text (Section 7), so we do not claim the channel switch alone as a gain. We build the harness ladder on JSON text because the harness can parse and inspect it directly.

State summary (+3.5 pp). The harness-written snapshot scores 68/200 ($p = 0.30$ against JSON text). Empty turns fall from 41 to 31 while state mismatches do not (60 to 64), suggesting that the summary mainly keeps the model calling.

Stop on repeats and the status card (minimal recipe, 35.5%). The gated stop rule scores 65/200 (+2.0 pp, $p = 0.48$). Stopping at the *first* repeat instead scores 54/200, 11 tasks below the gated rule, and all 37 forced stops fell on tasks that failed. Adding the status card gives the minimal recipe, 71/200 (+5.0 pp over JSON text, $p = 0.16$). Relative to stop on repeats alone, empty turns

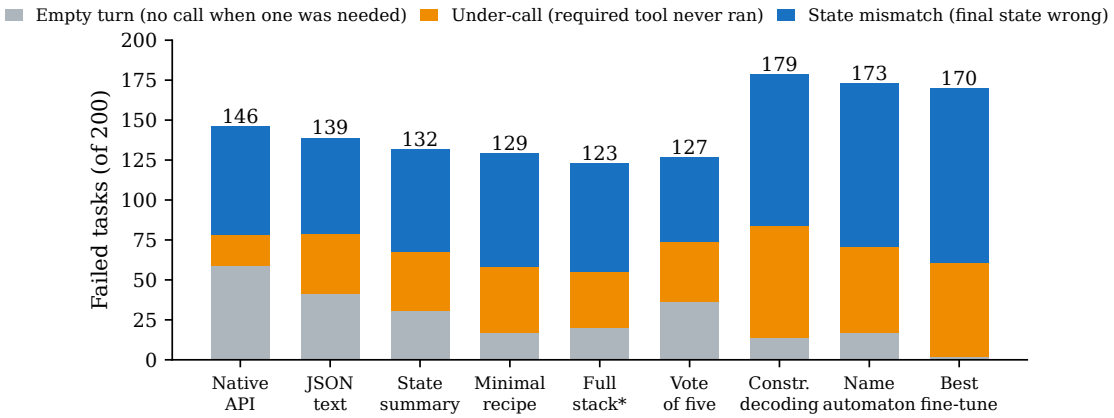


Figure 2: Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the headline full stack; Constr. decoding = constrained decoding; Name automaton = tool-name automaton; conditions as in Section 3); its height is the number of failed tasks, split by primary error label (Section 3). Harness control mostly removes empty turns (grey); wrong final states (blue) remain. Forced-valid decoders and the best fine-tune shift failures into under-calls (orange) and state mismatches.

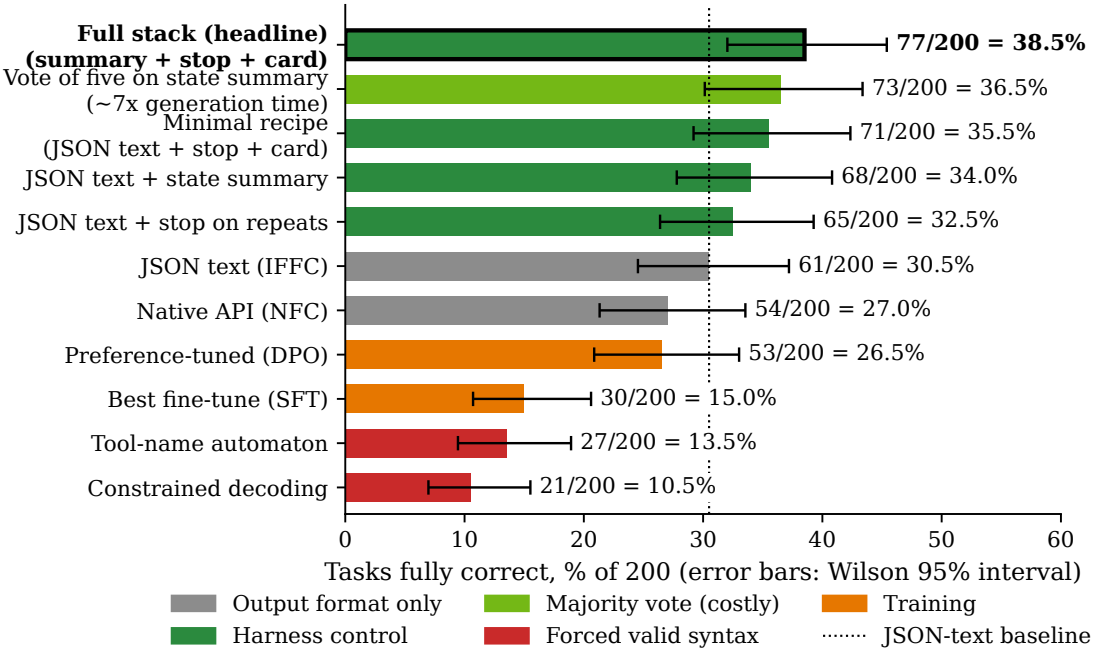


Figure 3: Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Dark green: harness control on JSON text (the headline full stack is outlined and in bold); light green: majority vote, the compute-heavy comparison; grey: output format alone; red: forced-valid decoders; orange: training. Dotted line: JSON-text baseline (30.5%).

fall from 38 to 17 while state mismatches rise from 60 to 71, so the error labels suggest the card acts mainly as an act-versus-abstain control.

Full stack: the best measured result (38.5%). Combining all four greedy changes scores 77/200 (38.5%, interval 32.0–45.4%). Paired against JSON text, it solves 30 tasks that JSON text misses and misses 14 that JSON text solves ($p = 0.023$ uncorrected; not significant after correction). Against the minimal recipe, the state summary alone and the vote of five, the differences are within noise (Table 1). The configuration was assembled after we had seen earlier results on these tasks, so its lead is subject to tuning leakage (Section 7). Its harness code is in pull request #90 (run at commit `c894ec6`, merged as `9aef7d7`).

Cost: majority vote (36.5%). Voting over five samples on top of the state summary scores 73/200, four tasks fewer than the full stack, which solves 24 tasks the vote misses and misses 20 it solves ($p = 0.65$). The vote takes about 7× as long to generate as the greedy full stack, from a single sampling seed. A greedy harness matches it at a fraction of the time, which matters on a local device.

How sure are we? Intervals for the harness rows are about ±6–7 pp wide (Table 4). Among harness conditions, only the full stack versus JSON text ($p = 0.023$) and the minimal recipe versus the native API ($p = 0.033$) fall below 0.05, and neither survives correction for multiple comparisons.

Table 1: Exact two-sided McNemar tests on per-task results (200 tasks each, same task IDs). "A only" and "B only" count tasks that one condition solves and the other misses. p is uncorrected; "Corrected" marks whether it stays below 0.05 after correcting for these eight tests (Holm's method). Full stack: JSON text + state summary + stop on repeats + status card; minimal recipe: JSON text + stop on repeats + card. Exploratory comparisons outside this family are in Table 3.

A	B	A only	B only	p	Corrected
Full stack (38.5%)	Minimal recipe (35.5%)	19	13	0.38	no
Full stack	Vote of five (36.5%)	24	20	0.65	no
Full stack	State summary (34.0%)	24	15	0.20	no
Full stack	JSON text (30.5%)	30	14	0.023	no
Minimal recipe	JSON text	26	16	0.16	no
Minimal recipe	Native API (27.0%)	37	20	0.033	no
JSON text	Native API	22	15	0.32	no
Best SFT (15.0%)	JSON text	7	38	< 0.001	yes

The ordering among the harness rows is therefore tentative, and the +8.0 pp gain is suggestive. Two contrasts are robust. The best supervised fine-tune solves only 7 tasks that JSON text misses while missing 38 that it solves ($p < 0.001$, surviving correction), and the full stack's interval (32.0–45.4%) does not overlap those of the best fine-tune (10.7–20.6%) or the forced-valid decoders. Preference tuning (26.5%) is *not* separable from JSON text.

6 Fine-Tuning Did Not Help

Fine-tuning is the standard way to adapt a small model to a task. Here no trained model beat the untuned model: the best supervised fine-tune scored 15.0%, significantly below JSON text, and the DPO model 26.5%, within noise of it. We treat the runs as evidence about *what* goes wrong; the version history is in Appendix B.1.

Public data taught the model to stop. The first mixes scored 1/200 (0.5%). A *stop target* is a training response that is an empty list ("no further call"). After a tool result, 78.1% of targets in that data were stop targets (2,121 versus 596 continue), all from ToolACE, so the model learned to stop after its first tool result while BFCL needs further calls. Removing those targets and adding recovery examples scored 30/200 (15.0%), our best fine-tune: empty turns nearly vanished (167 to 2), but failures moved to state mismatch (109) and under-call (59). Adding the state summary to this fine-tune lowered it to 26/200, although the summary helps the untuned model.

A rebuilt mix collapsed. The best-validation checkpoint of the rebuilt mix scored 2/20 on the gate (Figure 4); validation loss did not predict the collapse. The collapsed model usually made one call and stopped, even when a setup step was needed (looking up a stock symbol, changing directory, logging in), and invented missing values such as an order number. Without the card it scored 1/20, and continued training on test-style tool layouts still scored 2/20. To test the pipeline, we ran a *null test*, a fine-tune that should change nothing if the machinery is sound: the untuned model trained for one pass (39 updates) on 156 of its own logged answers through the same template, adapter, merge, quantization and serving steps. It scored 15/20 (interval 53–89%), within noise of the bar (17/20; 64–95%), while the rebuilt mix's 2/20 (3–30%) is far outside it. The null test rules out those pipeline steps as the cause at small update budgets. The rebuilt mix differs from it in both content

and training length (22,000 steps for the evaluated checkpoint), so its data are the leading suspect but not the only one.

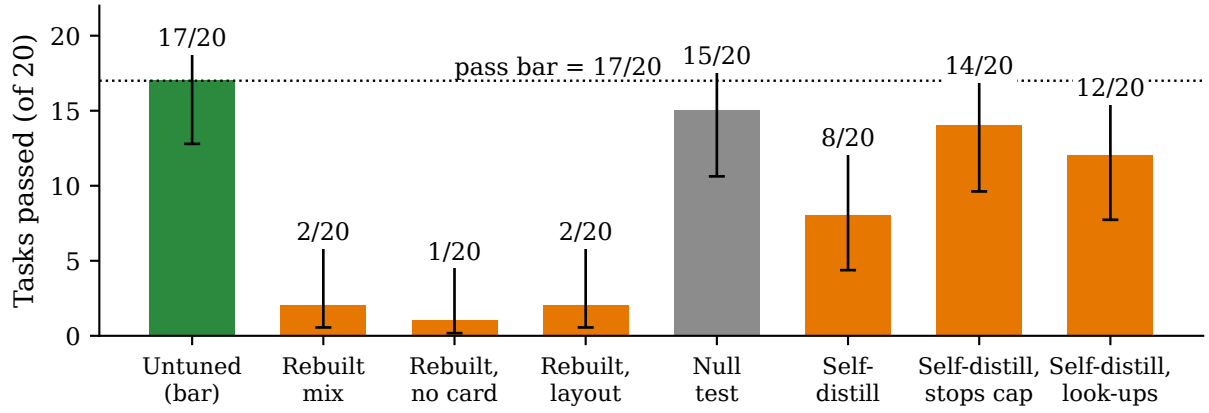


Figure 4: 20-task gate (the same 20 tasks for every bar; these are not 200-task accuracies). Error bars: Wilson 95% intervals, in tasks. Green: the untuned model with the status card, which sets the bar (17/20). Orange: fine-tuned models, run with the card except the one marked "no card". Grey: the null test. "Layout": continued training on test-style tool layouts; "stops cap" and "look-ups": retrains with the stop share capped or look-up episodes added. Every model trained on the rebuilt mix fails (1–2/20), while the null test stays close to the bar. Self-distillation (training on the model's own checker-approved answers to simulated tasks) and its two rebalanced retrains land in between.

A candidate cause that targeted retrains did not confirm. *Self-distillation*—training on the model's own answers to simulated tasks, keeping only turns the task checker marked correct—scored 8/20; in 6 of 12 failures the model said an ID or price was missing and stopped instead of calling the tool that would supply it. Compared with the null-test data, look-up-type calls fell from 37.9% to 19.7% of calls and stop targets rose from 23.1% to 33.2%. Two targeted retrains tested this: capping the stop share scored 14/20, and restoring the look-up share (39.6%) scored 12/20. Neither passed, so the data are *consistent with*, but do not establish, a shift in the decision mix (stop versus continue, look up versus act) as the cause. Leak checks found no evaluation task IDs or tool names in the null-test or self-distillation data.

7 Discussion and Limitations

State, not syntax. Across conditions, the hard part is choosing the next step from the current state of the world, including whether to stop; writing a valid call is the easy part. The levers that help support that decision: a summary of real results, a gated stop, and a card that mainly controls whether the model acts. None of them reduces state mismatches, which remain the largest failure bucket, so the gains come from acting at the right moments, and tracking state itself remains unsolved. Levers that constrain the surface form hurt, and training appeared to shift the balance between acting and stopping rather than improve tracking.

The guided-template tie. A seven-step reasoning template on the native API also scored 61/200, tying JSON text (19 tasks swap each way; state mismatches rose to 79). The +3.5 pp channel gap

may therefore reflect prompt structure as much as the channel ($p = 0.32$ in any case). Our claim is narrower: on top of JSON text, harness control adds +8.0 pp, a suggestive gain (Section 5). We did not test the harness levers on the native API or combined with the template.

What we recommend. For an on-device agent: JSON-text calls, a harness-written state summary, a gated stop rule and a status card. All four are greedy and need no training data or training compute; the minimal recipe (35.5%) is a smaller fallback. Before fine-tuning, screen on task-level checks rather than validation loss. Even the best configuration completes fewer than two tasks in five: this 2B model is not yet a reliable multi-step agent on this benchmark.

Limitations.

- **One model, one split, one device, tuned in place.** All multi-turn results use Gemma-4-E2B-it on the 200-task base split on one laptop. Settings (stop threshold, card fields, mix filters) were tuned and reported on the same tasks. Other models and the harder BFCL splits are untested.
- **Headline selection.** The full stack was assembled after seeing earlier results on the same 200 tasks (tuning leakage), and the correction family was fixed after that; its only nominally significant gain does not survive correction.
- **Sample size and decoding.** Intervals for the harness rows are about ±6–7 pp; greedy decoding with one quantization and one seed. The vote of five uses one sampling seed, and its cost was measured with four parallel slots.
- **Heuristic error labels.** Each failed task gets one label from string matches on the checker's message, so shifts between labels suggest, but do not prove, a mechanism.
- **Gate design.** The 20 gate tasks were picked post hoc using historical per-task results from the same split, and the bar was set by the configuration it screens against, so the gate is a screening heuristic.
- **Fairness of comparisons.** The native baseline used eight server slots versus four elsewhere; the card and the vote were never run on plain JSON text. Per-task results come from a local evaluation database that is not in the repository.
- **Home-made constrained decoding.** Ours is a two-pass approximation on stock llama-server, so "forcing valid syntax hurts" may partly mean "our approximation hurts".
- **Training confounds.** Mix variants changed several factors at once; self-distillation took about 3× as many updates as the null test, and the rebuilt mix far more.

8 Future Work and Conclusion

Future work. First, *confirmatory runs*: re-score the full stack and the JSON baseline on held-out tasks, the harder BFCL multi-turn splits and a second small model, with matched serving. Second, *retrieval-augmented tool selection*: retrieve the few relevant tools for each step instead of placing every schema in context [4, 25]. Third, *decision-balanced training data*: vary the look-up and stop shares with everything else fixed.

Conclusion. Gemma-4-E2B-it picks a single call well (83.1%) but completes only 27.0% of multi-turn tasks through the same native interface; its failures are mostly under-calls and wrong final states, and forcing valid output makes them worse. Four greedy harness changes reached 38.5%, on par with a five-sample vote at a seventh of the time. The +8.0 pp over JSON text is suggestive ($p = 0.023$ uncorrected; not significant after correction) and was found on the tasks used for tuning. Fine-tuning did not reach that level, and a null test points to the training data rather than the pipeline. For practitioners, the best configuration we measured is also far cheaper than voting or training.

References

[1] Google. Gemma 4 E2B instruct. `https://huggingface.co/google/gemma-4-e2b-it`, 2026. E2B = effective 2B.

[2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. `https://gorilla.cs.berkeley.edu/leaderboard.html`, 2024.

[3] ggml-org. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2024.

[4] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[5] Yalda Taheri, Mohammad Hassan Heydari, Erfan Naaman, and Afsaneh Fatemi. Small reasoning models are instruction followers in function calling. *arXiv preprint arXiv:2608.22472*, 2026.

[6] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[7] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Dynamic and efficient structured generation engine for agentic LLMs. *arXiv preprint arXiv:2601.04426*, 2026.

[8] M. Yu, K. Mu, and G. Wu. PTC-Decoder: Towards intelligent SLMs on offline resource-constrained edge devices. *arXiv preprint arXiv:2609.30836*, 2026.

[9] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? A study on the impact of format restrictions on performance of large language models. *arXiv preprint arXiv:2408.02442*, 2024.

[10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.

[11] Rongfeng Guo, Yinxuan Huang, Yusen Wu, Maoqing Zhong, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. From state to action: OODA-Tool for reliable multi-turn tool use. *arXiv preprint arXiv:2608.24368*, 2026.

[12] Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, et al. Improving large language models function calling and interpretability via guided-structured templates. *arXiv preprint arXiv:2509.18076*, 2025.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, et al. xLAM: A family of large action models to empower AI agent systems. *arXiv preprint arXiv:2409.03215*, 2024.

[15] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, et al. Robust function-calling for on-device language model via function masking. In *International Conference on Learning Representations (ICLR)*, 2025.

[16] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, et al. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*, 2024.

[17] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, et al. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *arXiv preprint arXiv:2406.18518*, 2024.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Sunghee Jung et al. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. *arXiv preprint arXiv:2504.02882*, 2025.

[21] Apple Machine Learning Research. MLX: An array framework for Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[22] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[24] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

A Intervention details

JSON text (IFFC). Following Appendix B of [5], the tool list is omitted from the API request; the system prompt asks for a JSON list of `{function_name, parameters}` objects in assistant text. Tool results return as user messages. Calls map to the same BFCL function strings as native calling.

State summary. After each successful tool execution, the harness writes a compact `[STATE]` block from real results (current directory, known files, last executed calls). The last two turns stay verbatim; older raw outputs are dropped. It is not a model-written summary. Example:

```
[STATE]
cwd: /workspace/notes
files: todo.txt, log.csv
last: cd(folder='notes')->ok
```

Stop on repeats. After a tool result: (1) if the proposed call set equals the previous one, stop; (2) else if every proposed call was already executed in this user request *and* at least six non-empty tool steps have occurred in it, stop; (3) otherwise execute. The six-step window resets on a new user request. The ungated variant applies rule (2) with no minimum.

Status card (tag `[OODA]`). Prefixed to a working copy of the latest user message (the stored transcript is unchanged), stacked on stop on repeats. Fields: `goal` (latest user question, truncated); `slots` (up to eight key–value facts from executed arguments and results); `last` (last executed call and outcome); `next` (advisory: `refuse` if no tools are available, `ask` if the last result signals a missing or invalid argument, `final` if the user's message is a closing "thanks/done", otherwise `call`). The hint does not constrain decoding and the card never reads reference trajectories. It is not the multi-pass OODA-Tool of [11]. Example:

```
[OODA]
goal: lock the doors and start the engine
slots: lockStatus=locked
last: startEngine()->error: brake pedal must be pressed
next: call
```

Full stack. The state summary, stop on repeats and status card together: the summary is written into the stored transcript after each executed call, and the card is prefixed to a working copy of the latest user message, as above.

Vote of five. On the state-summary configuration, each step samples five completions at temperature 0.7, majority-votes the `(function_name, arguments)` tuple, executes the winner once, and continues. It is not a learned verifier.

Constrained decoding. Two requests per call: the first constrained to an empty list or a list of valid tool names, the second constraining arguments with each tool's JSON schema. No native tool grammar is active. This approximates XGrammar-2's tag-triggered dispatch [7] on stock llama-server.

Tool-name automaton. A live finite automaton restricts the tool name to valid names while arguments are free, on JSON text, not stacked on stop on repeats. Execution accuracy here is not comparable to the F1 reported by Yu et al. [8].

Training. LoRA adapters [18] on Gemma-4-E2B-it with mlx-lm; best-validation checkpoint fused and quantized to Q4_K_M; evaluated with JSON text. The rebuilt mix: 30,000 training / 1,500 validation rows; 45/40/10/5% simulated / APIGen / ToolACE / xLAM; 22% empty-list targets (all after a tool result); 41.2% of rows with multi-call targets; no evaluation tool names copied into simulations. DPO: β=0.2, LoRA, scored on JSON text.

B Failure labels, exploratory tests and training history

Table 2: Primary error per failed task (200 tasks). NFC = native API; IFFC = JSON text; Stop = stop on repeats (gated); Ungated = stop on first repeat; Summary = state summary; Minimal = minimal recipe; Full = full stack; Vote = vote of five; CD = constrained decoding; Autom. = tool-name automaton; SFT = best fine-tune. Labels as defined in Section 3.

Label	NFC	IFFC	Stop	Ungated	Summary	Minimal	Full	Vote	CD	Autom.	SFT
Empty turn	59	41	38	40	31	17	20	36	14	17	2
Under-call	19	38	37	31	37	41	35	38	70	54	59
State mismatch	68	60	60	75	64	71	68	53	95	102	109
Failed	146	139	135	146	132	129	123	127	179	173	170

Table 3: Exploratory McNemar tests, outside the eight-test correction family (uncorrected p; columns as in Table 1).

A	B	A only	B only	p
State summary (34.0%)	JSON text (30.5%)	20	13	0.30
Vote of five (36.5%)	State summary	17	12	0.46
Stop on repeats (32.5%)	JSON text	11	7	0.48
Minimal recipe (35.5%)	Stop on repeats	26	20	0.46

B.1 Training history

Version history. Dropping only mid-dialogue empty targets from the first mix still scored 1/200; deleting trailing empty answers at serving time scored 4/200. After the repaired mix (30/200), two

further filtering variants scored 23/200 and 26/200 (Table 4); each changed several filters at once, so they are not single-factor ablations. The rebuilt mix's training was stopped early at step 26,500 of 32,000; the best validation loss (0.030) was at step 22,000, and that checkpoint is the one evaluated. Continued training for 2,500 steps on test-style tool layouts scored 2/20.

Self-distillation counts. Look-up-type calls: 67/177 in the null-test data versus 78/395 in the self-distillation data; stop targets: 36/156 versus 146/440. Capping the stop share (22.9%) left the look-up share low (20.6%). The look-up-restoring retrain lost 7 tasks the untuned model solves, and empty turns returned (9 of 66 turns). Look-up counts rest on a name-matching heuristic.

Stop targets in the rebuilt mix. Stop targets were 22% of all rebuilt-mix rows, all after a tool result, a different denominator from the first mix's 78.1% (the share of targets *after a tool result* that were stops). The rebuilt-mix models still under-called, so stop share alone does not explain the collapse.

10-task tripwire. Before the 20-task gate, trained models faced a disjoint 10-task check. Two fine-tuned checkpoints and the untuned model with the card all scored 3/10, passing the same three tasks, and across 11 strong historical 200-task runs the tripwire's rank correlation (Spearman) with the 200-task score was 0.118 (approximate 95% interval −0.52 to 0.67). We therefore used it only as a regression tripwire, never to rank runs.

C Full scoreboard, reproducibility and glossary

Reproducibility. Hardware: MacBook Pro, Apple M4 Pro, 48 GB. llama.cpp build 8111 (commit `11c325c6e`), `llama-server -jinja`, flash attention, full GPU offload, `-c 65536`, `-np 4` (native baseline `-np 8`). BFCL v4 via the `bfcl-eval` package (exact version per run not logged). Temperature 0 except voting (0.7, N=5, one seed). The full stack (ID 78 in Table 4) is `benchmark.db` `scores.id` 51 with log `sft/v13/paper_evals/runs/b_fullstack_mtb200.log`; its harness code is pull request #90, run at commit `c894ec6` and merged as `9aef7d7`. Single-call (ID 79) is `scores.id` 52–58 with log `sft/v13/paper_evals/runs/a_single_turn_gemma.log`. Failure labels are derived from the checker messages in the `eval_errors` table. Figures are regenerated from these counts by `paper/figures/src/make_refocus_figs.py`.

Glossary.

AST Abstract syntax tree; the single-turn BFCL matcher.
BFCL Berkeley Function Calling Leaderboard.
BoN Best-of-N; internal name for the vote of five.
DPO Direct Preference Optimization.
Full stack JSON text + state summary + stop on repeats + status card.
Greedy decoding Always picking the most likely next token (temperature 0); deterministic.
Harness The program around the model that parses calls, runs tools and records state.
IFFC Instruction-followed function calling: calls written as JSON in ordinary text; called *JSON text* in this paper.
LoRA Low-rank adapters: small trainable matrices added to frozen weights.
McNemar test A paired test using only tasks where two conditions disagree.
Minimal recipe JSON text + stop on repeats + status card.
NFC Native function calling: structured `tool_calls` via the server's tool API; called *native API* in this paper.
Null test A training run that should change nothing if the pipeline is sound.
OODA Observe–orient–decide–act; here only the tag printed on the status card.
pp Percentage points.
Q4_K_M A 4-bit weight-quantization format of llama.cpp.
Self-distillation Training the model on its own answers to simulated tasks that a checker marked correct.
SFT Supervised fine-tuning.
Slot One request a server processes concurrently with others.
State summary Harness-written snapshot of facts from executed results.
Status card Four-line card prefixed to the latest user message.
Stop on repeats Ends a turn when the model re-proposes executed calls.
Stop target A training response that is an empty list: no further call.

Tool-name automaton Decoder constraining only the tool name.
Turn One user request and everything the model does before handing control back.
Vote of five Majority vote over five sampled calls per step.
Wilson interval A confidence interval for a proportion that stays accurate for small counts.
10-task tripwire Fixed 10-task regression check; does not rank models.
20-task gate Fixed 20-task screening check (bar 17/20); not a 200-task accuracy.

Table 4: Every scored condition in this paper (Gemma-4-E2B-it Q4_K_M), with its internal evaluation-log ID (a private local database) and internal name in parentheses. Interval: Wilson 95%. The 20-task rows are screening checks, not 200-task accuracies.

ID	Condition	Result	Acc.	95% interval
Multi-turn base, 200 tasks				
78	Full stack: state summary + stop + card (headline; `iffc-state-ooda`, PR #90)	77/200	38.5%	32.0–45.4
11	Vote of five on state summary (BoN N=5 on IFFC+STATE)	73/200	36.5%	30.1–43.4
42	Minimal recipe: card on stop on repeats ("R42")	71/200	35.5%	29.2–42.3
10	State summary (IFFC+STATE)	68/200	34.0%	27.8–40.8
30	Stop on repeats, gated at six steps (MIN=6)	65/200	32.5%	26.4–39.3
5	JSON text (IFFC)	61/200	30.5%	24.5–37.2
9	Guided template on native API (GST+NFC)	61/200	30.5%	24.5–37.2
4	Native API (NFC)	54/200	27.0%	21.3–33.5
27	Stop on first repeat (ungated)	54/200	27.0%	21.3–33.5
51	DPO, full train	53/200	26.5%	20.9–33.0
15	Best SFT, repaired mix (SFT-v4)	30/200	15.0%	10.7–20.6
46	Tool-name automaton (PTC-A)	27/200	13.5%	9.4–18.9
18	SFT, filters on first-turn targets off† (SFT-v6)	26/200	13.0%	9.0–18.4
16	Best SFT + state summary (SFT-v4+STATE)	26/200	13.0%	9.0–18.4
17	SFT, extra first-turn and state filters† (SFT-v5)	23/200	11.5%	7.8–16.7
8	Constrained decoding, two-pass (CD)	21/200	10.5%	7.0–15.5
14	SFT first mix, trailing empty answers stripped at serving (SFT-v2 strip)	4/200	2.0%	0.8–5.0
12, 13	SFT first mixes, two variants (SFT-v1, SFT-v2)	1/200 each	0.5%	0.1–2.8
20-task gate (same 20 tasks for every row)				
69	Untuned + card: gate bar	17/20		
70	Rebuilt mix, best-validation checkpoint (SFT-v11, step 22,000)	2/20		
71	Rebuilt mix, no card	1/20		
72	Rebuilt mix, layout continue-train (SFT-v11 fmt2500)	2/20		
73	Null test (v13 null)	15/20		
74	Self-distillation, correct turns only (v13-sd)	8/20		
75	Self-distillation, stop share capped (v13-sdb)	14/20		
76	Self-distillation + simulated look-up episodes (v13-lk)	12/20		
Single-call BFCL v4 categories, native API‡				
79	All five categories below (weighted)	823/990	83.1%	80.7–85.3
	Simple	375/400	93.75%	
	Multiple	183/200	91.50%	
	Irrelevance	214/240	89.17%	
	Java	37/100	37.00%	
	JavaScript	14/50	28.00%	

†First-turn filters drop or require training examples where the first response is empty or under-specified; state filters drop continuations inconsistent with executed tool results. ‡The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output.